

SCS 1309

Database Management Systems

Lecture 6 – Structured Query Language (SQL)

MVPT Lakshika,
Lecturer (Probationary),
Department of Information Systems Engineering, UCSC.
Contact: tlv@ucsc.cmb.ac.lk

Outline

- Introduction to Triggers
- Create, Drop, and Show Triggers
- Types of Triggers

Introduction

- **Triggers** can be used to specify **automatic actions** that the database system will perform when **certain events occur** and **conditions are satisfied**.
- This type of functionality is generally referred to as **active databases**.

Example: A manager may want to be informed if an employee's travel expenses exceed a certain limit by receiving a message whenever this occurs.

- Specify the **condition** that, if violated, causes some user to be informed of the violation.
- The **action** that the DBMS must take in this case is to **send an appropriate message to that user**.
- The *condition* is thus used to monitor the database.
- Other *actions* may be specified.

Example: executing a specific *stored procedure* or triggering other updates.

Create Triggers

The basic syntax:

```
CREATE TRIGGER trigger_name
{ BEFORE | AFTER | INSTEAD OF } { INSERT | UPDATE | DELETE }
ON table_name
[ FOR EACH ROW ]
BEGIN
    -- Trigger body or Trigger logic (SQL statements) here
END;
```

BEFORE, AFTER, INSTEAD OF: Specifies [when the trigger should be executed](#) (INSTEAD OF triggers are typically used in SQL Server, Oracle, PostgreSQL, but not supported in MySQL)

INSERT, UPDATE, DELETE: Specifies the [type of operation/event that activates the trigger](#).

FOR EACH ROW: Indicates the [row-level trigger](#).

BEGIN and END: Bound the trigger body, defining the SQL statements to be executed.

Create Triggers

A typical trigger, which is regarded as an **ECA (Event, Condition, Action)** rule, has **three components**:

1. **Event(s)**: These are usually database update operations that are explicitly applied to the database. These events are specified after the keyword **{BEFORE|AFTER|INSTEAD OF}**.

Example events:

- INSERT, UPDATE, or DELETE statement on a specified table (or possibly view)
 - CREATE, ALTER, or DROP statement on any schema object
 - A database startup or instance shutdown, or a user logon or logoff
 - Any or specific error messages
- Oracle also supports **INSTEAD-OF** triggers, which provide a transparent way of modifying views that cannot be modified directly through SQL DML statements (INSERT, UPDATE, and DELETE).

Create Triggers

2. Condition: Determines whether the **rule action** should be executed. Once the triggering event has occurred, an **optional** condition may be evaluated.

- If ***no condition is specified***, the action will be executed once the event occurs.
- If **a condition is specified**, it is first evaluated, and only *if it evaluates to true*, the rule action will be executed.
- The condition is specified in the **WHEN** (PostgreSQL or Oracle) clause of the trigger or **IF-THEN-ELSE** (MySQL) clause of the trigger.

3. Action to be taken: The action is usually a **sequence of SQL statements**.

- Action could also be a database transaction or an external program that will be **automatically executed**.

Example 1

DreamHome has a rule that prevents a member of staff from managing more than 10 properties at the same time.

```
DELIMITER //
CREATE TRIGGER StaffNotHandlingTooMuch
BEFORE INSERT ON PropertyForRent
FOR EACH ROW
BEGIN
    DECLARE vpCount INT;
    SELECT COUNT(*) INTO vpCount FROM PropertyForRent WHERE staffNo = NEW.staffNo;

    IF vpCount > 10 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = CONCAT('Member ', NEW.staffNo, ' already managing 10 properties');
    END IF;
END //
DELIMITER ;
```

Example 1 (Cont...)

DreamHome has a rule that prevents a member of staff from managing more than 10 properties at the same time.

```
DELIMITER //
CREATE TRIGGER StaffNotHandlingTooMuch
BEFORE UPDATE ON PropertyForRent
FOR EACH ROW
BEGIN
    DECLARE vpCount INT;
    SELECT COUNT(*) INTO vpCount FROM PropertyForRent WHERE staffNo = NEW.staffNo;

    IF vpCount > 10 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = CONCAT('Member ', NEW.staffNo, ' already managing 10 properties');
    END IF;
END //
DELIMITER ;
```

Example 1 (Cont...)

- This trigger is invoked before a row is **inserted into** the *PropertyForRent* table or **an existing row is updated**.
- If the member of staff currently manages 10 properties, the system displays a message and aborts the transaction.
- The following points should be noted:
 - The **BEFORE** keyword indicates that the trigger should be executed before an insert or update is applied to the *PropertyForRent* table.
 - The **FOR EACH ROW** keyword indicates that this is a *row-level trigger*, which executes for each row of the *PropertyForRent* table that is updated in the statement.
 - The **new** keyword is used to refer to the new value of the column(Although not used in this example, the **old** keyword can be used to refer to the old value of a column.)

Example 1(Cont...)

- MySQL does not allow combining two events (INSERT and UPDATE) into a single CREATE TRIGGER statement.
- You must create two separate triggers if you want the same logic to apply to both actions.
- Best Practice: If your logic is very complex and you don't want to copy-paste the same code into two different triggers (which makes it hard to maintain), you should use a **Stored Procedure**.

Step 1: Create a Procedure that contains the logic.

Step 2: Call that Procedure from both triggers.

Example 1 (Cont...)

Step 1: Create a Procedure

```
DELIMITER //  
CREATE PROCEDURE CheckStaffPropertyLimit(IN p_staffNo  
VARCHAR(10))  
BEGIN  
    DECLARE vpCount INT;  
    SELECT COUNT(*) INTO vpCount  
    FROM PropertyForRent  
    WHERE staffNo = p_staffNo;  
    IF vpCount >= 10 THEN  
        SIGNAL SQLSTATE '45000'  
        SET MESSAGE_TEXT = 'Staff member already managing  
10 properties';  
    END IF;  
END //  
DELIMITER ;
```

Step 2: Create a trigger to call procedure

```
CREATE TRIGGER StaffLimit_BeforeInsert  
BEFORE INSERT ON PropertyForRent  
FOR EACH ROW  
CALL CheckStaffPropertyLimit(NEW.staffNo);
```

```
CREATE TRIGGER StaffLimit_BeforeUpdate  
BEFORE UPDATE ON PropertyForRent  
FOR EACH ROW  
CALL CheckStaffPropertyLimit(NEW.staffNo);
```

* If your trigger body has only **one** command (e.g., CALL statement), BEGIN...END is **optional**.

Example 1 (Cont...)

NEW and **OLD** keywords:

- In an **INSERT**, only the **NEW** keyword exists.
- In an **UPDATE**, both **OLD** and **NEW** keywords exist.
- In a **DELETE**, only the **OLD** keyword exists.

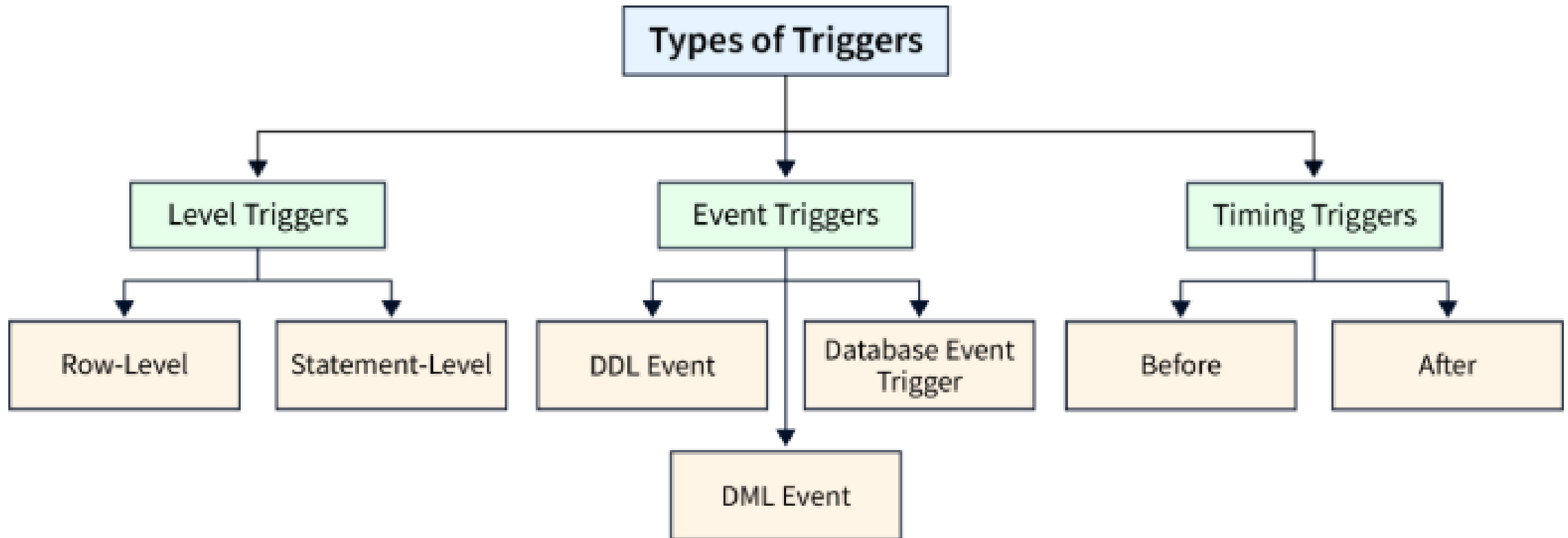
Example: If you **only** want to check the staff limit **if the staff member actually changes**, you use both keywords together:

```
CREATE TRIGGER StaffLimit_BeforeUpdate
BEFORE UPDATE ON PropertyForRent
FOR EACH ROW
CALL CheckStaffPropertyLimit(NEW.staffNo);
```

Modified

```
CREATE TRIGGER StaffLimit_BeforeUpdate
BEFORE UPDATE ON PropertyForRent
FOR EACH ROW
BEGIN
  IF (NEW.staffNo <> OLD.staffNo) THEN
    CALL CheckStaffPropertyLimit(NEW.staffNo);
  END IF;
END;
```

Types of Triggers



Types of Triggers: Level Triggers

- The SQL standard defines **two types of triggers**:
 - **A row-level trigger:** executes **for each row** of the table that is affected by the triggering event.
Example: if a table has 100 rows inserted, updated, or deleted, the trigger is automatically invoked 100 times for the 100 rows affected.
 - **A statement-level trigger:** execute **only once**, even if multiple rows are affected by the triggering event.
- **MySQL supports only row-level triggers.** It doesn't support statement-level triggers.

Types of Triggers: Event Triggers

DDL triggers

- Events related to the Data Definition Language (DDL) commands, such as [Create_table](#), [Create_view](#), [drop_table](#), [Drop_view](#), and [Alter_table](#), cause the DDL triggers to be activated.
- DDL triggers allow tracking [changes in the structure of the database](#).
- DDL triggers are not supported in MySQL.

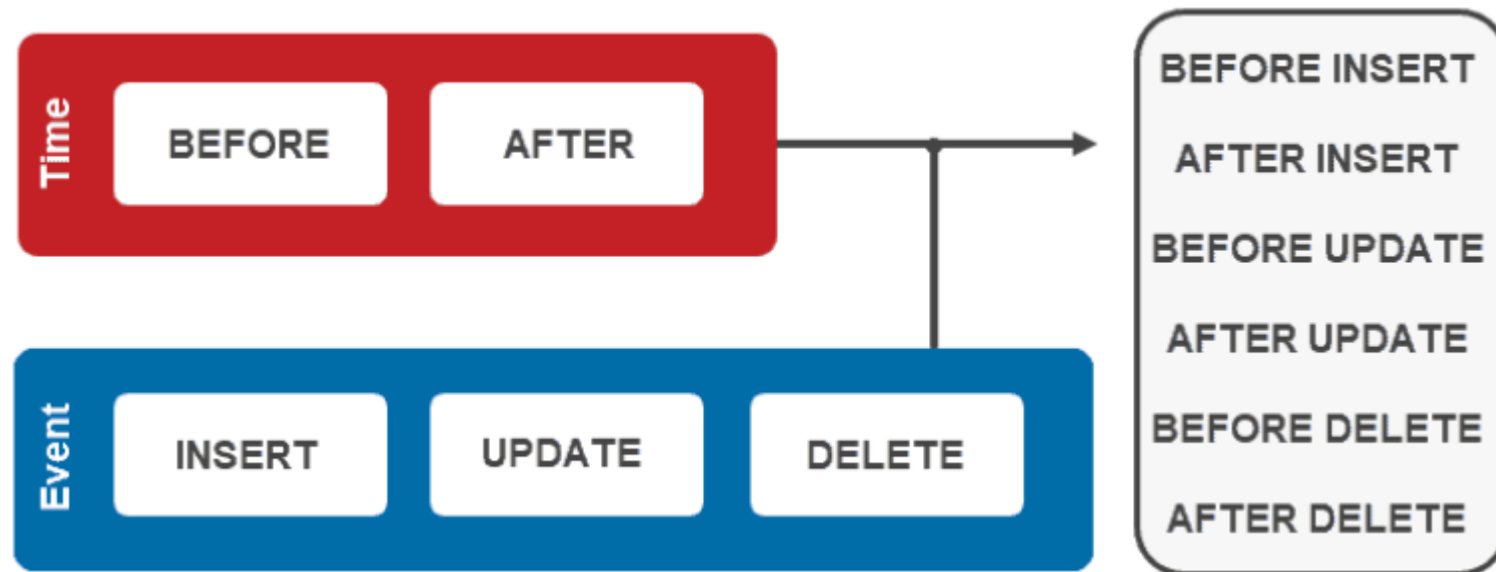
```
CREATE TRIGGER prevent_table_creation ON DATABASE
FOR CREATE_TABLE, ALTER_TABLE, DROP_TABLE
AS
BEGIN
PRINT 'you can not create, drop and alter table in this
database';
ROLLBACK;
END;
```

This is SQL Server Syntax. MySQL does not support this.

Types of Triggers: Event Triggers

DML triggers

- Events related to Data Manipulation Language (DML) commands, such as [Insert](#), [Update](#), and [Delete](#), cause the DML triggers to be activated.
- DML triggers are used for data validation, ensuring that modifications to a table are done under controlled conditions.



Types of Triggers: Event Triggers

Database Event triggers

- These typically refer to triggers that fire on **system-level events** rather than data changes.

Examples: Server Startup/Shutdown

User Login/Logoff

Database Errors

Types of Triggers: Event Triggers

Example: Logon Triggers

- These triggers are fired in response to logon events.
- Logon triggers are useful for
 - Monitoring user sessions or restricting user access to the database.
 - Track login activity or set a limit on the number of sessions that a given login can have to audit and manage server sessions.

```
CREATE TRIGGER track_logon ON LOGON
AS
BEGIN
PRINT 'A new user has logged in.';
END;
```

This is SQL Server Syntax. MySQL does not support this.

The PRINT statement messages and any errors generated by the trigger will all be visible in the Server error log.

More examples

BEFORE INSERT TRIGGERS

The Scenario: Preventing Negative Prices in the table *Products*.

```
DELIMITER //  
CREATE TRIGGER Check_Price_Before_Insert  
BEFORE INSERT ON Products  
FOR EACH ROW  
BEGIN  
    IF NEW.price <= 0 THEN  
        SIGNAL SQLSTATE '45000'  
        SET MESSAGE_TEXT = 'Error: Product price must be a  
positive value.';  
    END IF;  
END //  
DELIMITER ;
```

BEFORE INSERT trigger is used to automatically modify or validate data **before** it is permanently saved to the database.

If you try to run this:

```
INSERT INTO Products (productName, price)  
VALUES ('Wireless Mouse', -15.00);
```

The Result: The database will **reject** the row and return:

```
Error Code: 1644. Error: Product price must be a positive value.
```

More examples

An **AFTER INSERT** trigger is used to perform an action **after** a row has been successfully saved to the database.

AFTER INSERT TRIGGERS

The Scenario: Every time a new product is added to the *product* table, you want to automatically record that event in a separate *AuditLog* table so you have a history of all additions.

1. Create the log table to store the history

```
CREATE TABLE ProductAudit (  
    auditID INT AUTO_INCREMENT PRIMARY KEY,  
    productID INT,  
    actionPerformed VARCHAR(50),  
    actionDate TIMESTAMP DEFAULT  
CURRENT_TIMESTAMP  
);
```

2. Create the Trigger

```
DELIMITER //  
CREATE TRIGGER Log_New_Product  
AFTER INSERT ON Products  
FOR EACH ROW  
BEGIN  
    INSERT INTO ProductAudit (productID, actionPerformed)  
    VALUES (NEW.productID, 'NEW PRODUCT ADDED');  
END //  
DELIMITER ;
```

More examples

BEFORE UPDATE TRIGGERS

The Scenario: Imagine a rule where a product's price can never be lowered by more than 50% in a single update. If someone tries to change a \$100 item to \$10, the database should block it.

```
DELIMITER //  
CREATE TRIGGER Protect_Price_Drop  
BEFORE UPDATE ON Products  
FOR EACH ROW  
BEGIN  
    IF NEW.price < (OLD.price * 0.5) THEN  
        SIGNAL SQLSTATE '45000'  
        SET MESSAGE_TEXT = 'Error: Price cannot be reduced by  
more than 50% in one update.';  
    END IF;  
END //  
DELIMITER ;
```

A **BEFORE UPDATE** trigger inspects data that is already in the database (OLD) and compares it to the data the user is trying to save (NEW).

If you try to run this:

```
INSERT INTO Products (pID, price) VALUES (1, 30);
```

```
UPDATE Products SET price = 10 WHERE pID = 1;
```

The Result: If the condition is met, the SIGNAL command kills the transaction, and the old price remains unchanged.

9 23:39:25 UPDATE Product SET price = 2 WHERE pID = 1 Error Code: 1644. Error: Price cannot be reduced by more than 50% in one update.

More examples

An **AFTER UPDATE** trigger fires once the data has already been saved to the table.

AFTER UPDATE TRIGGERS

The Scenario: Record every price modification in a table called *PriceHistory*.

1. Create the log table to store the history

```
CREATE TABLE PriceHistory (  
    historyID INT AUTO_INCREMENT PRIMARY KEY,  
    productID INT,  
    oldPrice DECIMAL(10,2),  
    newPrice DECIMAL(10,2),  
    changedAt TIMESTAMP DEFAULT  
CURRENT_TIMESTAMP  
);
```

2. Create the Trigger

```
DELIMITER //  
CREATE TRIGGER Log_Price_Change  
AFTER UPDATE ON Products  
FOR EACH ROW  
BEGIN  
    -- Only log if the price was actually modified  
    IF OLD.price <> NEW.price THEN  
        INSERT INTO PriceHistory (productID, oldPrice,  
newPrice)  
        VALUES (NEW.productID, OLD.price, NEW.price);  
    END IF;  
END //  
DELIMITER ;
```

More examples

A **BEFORE DELETE** trigger fires immediately after a DELETE command is issued, but **before** the row is actually removed from the database.

BEFORE DELETE TRIGGERS

The Scenario: Consider the rule that a staff member cannot be deleted from the *Staff* table if they are currently assigned to manage at least one property in the *PropertyForRent* table.

```
DELIMITER //
CREATE TRIGGER Prevent_Active_Staff_Delete
BEFORE DELETE ON Staff
FOR EACH ROW
BEGIN
    DECLARE propertyCount INT;
    SELECT COUNT(*) INTO propertyCount FROM PropertyForRent WHERE staffNo = OLD.staffNo;
    IF propertyCount > 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Cannot delete staff: They are currently managing properties.';
    END IF;
END //
DELIMITER ;
```

More examples

An **AFTER DELETE** trigger executes immediately after a row has been removed from the database.

AFTER DELETE TRIGGERS

The Scenario: When a staff member leaves the company and is deleted from the *Staff* table, automatically move their information into a *StaffArchive* table.

1. Create the archive table

```
CREATE TABLE StaffArchive (  
    staffNo VARCHAR(5) PRIMARY KEY,  
    name VARCHAR(50),  
    position VARCHAR(50),  
    deleteDate TIMESTAMP DEFAULT  
    CURRENT_TIMESTAMP  
);
```

2. Create the Trigger

```
DELIMITER //  
CREATE TRIGGER Archive_Deleted_Staff  
AFTER DELETE ON Staff  
FOR EACH ROW  
BEGIN  
    INSERT INTO StaffArchive (staffNo, name, position)  
    VALUES (OLD.staffNo, OLD.fName, OLD.position);  
END //  
DELIMITER ;
```

Drop Triggers

```
DROP TRIGGER [IF EXISTS] [schema_name.]trigger_name;
```

IF EXISTS: Conditionally drop the trigger if the trigger exists (This clause is optional).

[schema_name.]: Specify the name of the schema to which the trigger belongs. If you skip the schema name, the statement will drop the trigger in the current database.

Show Triggers

Shows triggers for the **current database** you are using.

```
SHOW Triggers;
```

Shows triggers in a **specific database** without switching to it

```
SHOW TRIGGERS FROM [schema_name] ;
```

Advantages of Triggers

- Triggers provide another way to check the **integrity of data**.
- Triggers **handle errors** from the database layer.
- Triggers give an alternative way to **run scheduled tasks**.
 - By using triggers, you don't have to wait for the scheduled events to run because the triggers are invoked automatically before or after a change is made to the data in a table.
- Triggers can be useful for **auditing the data changes** in tables.

Disadvantages of Triggers

- Triggers can only **provide extended validations**, not all validations. For simple validations, you can use the NOT NULL, UNIQUE, CHECK, and FOREIGN KEY constraints.
- Triggers can be **difficult to troubleshoot** because they execute automatically in the database, which may not be visible to the client applications.
- Triggers may **increase the overhead**.

Any Questions?